

Now I have everything I need. Here's the full analysis:

mylilshopper — AI Pipeline Technical Analysis

Layer 1 — Simple (Non-Technical)

mylilshopper builds a detailed taste profile for each user by analyzing photos of their wardrobe and Pinterest boards using AI vision — extracting colors, fit preferences, style categories, and formality levels from every image. When a user asks for outfit recommendations, the system translates their taste profile into precise search queries, fetches real products, scores each one against their profile, and uses AI to assemble coherent outfit combinations. Every approval, rejection, and swap they make updates their profile in real time, so the system gets more accurate with every interaction. The key insight is that the system doesn't wait for you to describe your style — it reads it directly from your photos. This means it gets smarter passively, just from what you already own and what you already save.

Layer 2 — Technical Walkthrough

Step 1: Data Ingestion

Sources:

- **Wardrobe photos** — uploaded via `StyleVault.jsx`, compressed client-side (HEIC→JPEG via `heic2any`), stored as base64 data URLs directly in `wardrobe_items.image_url`
- **Pinterest boards** — scraped via `tools/scrape_public_images.js` using three fallback strategies: RSS feed → HTML scrape with JSON-LD extraction → SerpAPI fallback. Pinterest images proxied to Supabase Storage because the Pinterest CDN blocks bot fetch requests.
- **Onboarding answers** — stored in `style_profiles` (budget tier, stores, style tags, sizes)
- **User messages** — chat input to `api/agent.js` via POST

Why base64 everywhere: Anthropic's inference servers can't reliably reach third-party CDNs (Pinterest, Shopify). `analyzeImageStyle()` always fetches the image itself and converts to base64 before sending to Claude.

Step 2: Feature Extraction — `tools/analyze_image_style.js`

Model: Claude Haiku 4.5 (Vision)

Every image — wardrobe or aspiration — runs through a single structured Claude Vision call. The prompt is strict and constrained: it demands a specific JSON schema and explicitly prohibits fabrication (`null` over guessing).

Extracted per image:

```
item_type      → top / bottom / shoes / outerwear / dress / accessory / full_outfit
dominant_colors → [hex, hex, hex]      (sampled from the actual image)
secondary_colors → [hex, hex]
fit_type       → slim | relaxed | oversized | tailored | fitted | null
fabric         → string or null
formality_score → integer 1-10
style_category  → streetwear | minimalist | smart_casual | coastal | ... | null
brand          → string or null (only if visible)
occasion_suitability → [casual, work, evening, ...]
individual_items → breakdown array if full_outfit photo
confidence     → high | medium | low
```

Batched in groups of 3 via `Promise.all` in `api/analyze.js`. Each result is written back to `wardrobe_items` or `aspiration_items` in Supabase.

Step 3: Style DNA Synthesis — `tools/synthesize_style_dna.js`

Triggered after every wardrobe or Pinterest analysis batch completes. This is the statistical aggregation layer that turns a bag of image attributes into a single coherent user profile.

Algorithm: Recency-Weighted Frequency Counting

```
weight(item) =
  2.0   if uploaded/analyzed within last 30 days
  1.0   if 30-180 days
  0.5   if older than 180 days
```

For each attribute, a frequency map is built:

```
buildFrequencyMap(items, getter, weightFn)
→ { attribute_value: cumulative_weight, ... }
```

What gets computed:

Attribute	Method
<code>primary_colors</code>	Top 5 by weighted frequency, using <code>colors[0:2]</code> per item
<code>secondary_colors</code>	Top 3, using <code>colors[2:4]</code> per item
<code>dominant_fit</code>	Mode of the weighted fit distribution
<code>fit_consistency_score</code>	<code>max_weight / total_weight × 100</code> — measures how concentrated the distribution is
<code>primary_style_category</code>	Mode of weighted style distribution
<code>formality_range_min/max</code>	Weighted mean ± 1.5, clamped to [1,10]
<code>brand_affinities</code>	Top 10 brands by weighted frequency
<code>aspiration_gap</code>	Style categories in <code>aspiration_items</code> NOT present in <code>wardrobe_items</code>
<code>overall_confidence_score</code>	< 20 images = low, 20-50 = medium, 50+ = high

Result is **upserted** (not appended) to `style_dna` — one row per user, rebuilt fresh on every synthesis run.

Step 4: Product Search — `tools/search_products.js`

API: SerpAPI Google Shopping engine.

The agent constructs style-aware queries using DNA attributes before calling this tool:

```
[fit] [primary_color_1] [primary_color_2] [style_category] [item_type] [occasion]  
e.g. "relaxed navy camel minimalist trousers casual"
```

SerpAPI returns raw product data: `title`, `price`, `source (store)`, `thumbnail`, `link`, `brand`.
Crucially, `colors`, `fit_type`, and `style_category` all come back as `null` — they must be inferred downstream.

Step 5: Product Scoring — `tools/scoreProductMatch()`

A pure, synchronous, rule-based linear additive model. Baseline score: **50**.

Scoring table:

Signal	Condition	Delta
Color	Primary palette match	+25
Color	Secondary palette match	+15
Color	Avoided color	-40
Fit	Matches dominant fit	+20
Fit	In rejected fits	-35
Style	Primary category match	+20
Style	Secondary category match	+10
Brand	In affinity list	+10
Brand	In rejection list	-20
Price	Within 1.2× typical spend	+10
Price	Over 2× typical spend	-20

Color comparison: Since SerpAPI products lack hex colors, the scorer uses two paths:

- Product has `colors` array → convert hex to named bucket via `hexToBucket()` (HSV-based), compare bucket names
- Product lacks colors → extract color words from title via `extractTitleColors()` regex

Attribute inference from title:

```
inferFit("Relaxed Wide Leg Trousers") → "relaxed"  
inferStyle("Linen Resort Shirt") → "coastal"
```

Thresholds: Score ≥ 60 → passes to outfit builder. Score ≥ 85 → flagged high-confidence. If fewer than 3 products pass the 60 threshold, the top-scored products are returned anyway (fallback to avoid empty results).

Step 6: Outfit Assembly — `tools/build_outfits.js`

Model: Claude Haiku 4.5 (text only — no product images at this stage)

Claude receives:

1. The user's Style DNA context (primary style, fit, colors, aspiration gap)
2. A text summary of scored products: `name | price | score | style | fit`

It reasons about 3 outfit combinations subject to hard constraints in the prompt:

- Minimum: top + bottom + shoes (or dress + shoes)
- Total price $\leq$ budget
- Distinct color story per outfit
- Formality coherence
- At least one outfit addressing the aspiration gap

Claude returns a JSON array. The function then does **fuzzy product matching** to re-link Claude's named items back to full product objects (which have `image_url`, `product_url`, etc.):

```
function matchProduct(searchName, claudeName) {  
  // Tries substring match in both directions,  
  // then falls back to significant word overlap (>2 words of length >3)  
}
```

Finally, each resolved product is run through `checkOutfitMultiplier()` to compute how many existing wardrobe items it pairs with — this becomes the `wardrobe_multiplier` score shown in the UI.

Step 7: The Agentic Loop — `api/agent.js`

Model: Claude Haiku 4.5

Max turns: 4

Tool parallelism: all tool calls within a turn run via `Promise.all`

Available tools:

- `search_products` — calls SerpAPI + scorer
- `build_outfits` — calls Claude outfit assembly
- `create_order` — wallet check + order record
- `update_style_dna` — logs feedback signal

Critical implementation detail — product cache:

Claude's context window only receives truncated product text (name, price, score). Full product objects (including `image_url`, `product_url`) are stored in `productCache{}` keyed by category. When `build_outfits` is called, the agent merges Claude's category list with the cached objects so no data is lost.

Tool choice enforcement:

```
toolChoice = () =>  
  hasSearchResults && !lastOutfits ? { type: 'any' } : { type: 'auto' }
```

After search results exist but before outfits are built, Claude is forced to call a tool — preventing it from writing a text reply mid-pipeline.

The **system prompt** embeds the complete Style DNA as text, so no tool call is needed to fetch the profile — it's preloaded.

Step 8: Feedback Loop — `tools/update_style_dna.js`

Every user interaction feeds back into the profile:

Signal	What gets written
<code>approval</code>	Brand added to <code>brand_affinities</code> ; price updates <code>per_category_price_sensitivity</code> via weighted average: $(\text{existing} \times 2 + \text{new}) / 3$
<code>rejection</code>	Colors appended to <code>avoided_colors</code> (deduplicated, max 10); fit added to <code>explicit_dislikes.fits</code>
<code>post_delivery_positive</code>	Same as approval, intended for 2× weight (declared but not applied in current code)
<code>swap</code>	Handler exists but is currently a no-op

Threshold promotion (`_promoteConfirmedPreferences`): Scans all rejection/swap signals for the user. Any fit rejected ≥ 3 times is promoted to `explicit_dislikes.fits` as a confirmed preference. This is the only mechanism that promotes weak signals to hard preferences.

Layer 3 — Deep Dive (For Engineers)

Algorithms and Why They Were Chosen

Recency-weighted frequency counting (`synthesize_style_dna`)

A step-function decay (2.0 / 1.0 / 0.5) rather than exponential decay. This is appropriate here because:

- The data is sparse (most users have <50 images)
- Exponential decay with sparse data produces noisy gradients
- Step-function is interpretable and debuggable
- The 30-day / 180-day boundaries map to meaningful fashion time horizons (seasonal transitions)

Tradeoff: Step-functions create discontinuities. An item uploaded 29 days ago weights 4× more than one uploaded 31 days ago — this can cause the DNA to shift sharply around boundaries if upload dates cluster at those edges.

Linear additive scoring (`score_product_match`)

The scorer is deliberately not a learned model. Reasons:

- No labeled training data exists yet (too early to train a ranker)
- A learned model would be a black box — the `breakdown` array provides full explainability for every score, which feeds the agent's natural-language justifications

- The additive structure means the maximum possible score is bounded and predictable (max theoretical: $50 + 25 + 15 + 20 + 20 + 10 + 10 = 150$, clamped to 100)

The baseline of 50 means even a product with zero attribute matches doesn't score zero — it stays in the "maybe" range, which prevents the fallback from triggering too aggressively.

Hex-to-bucket color comparison

Converting hex colors to ~20 named buckets is a lossy transformation. The HSV-based `hexToBucket()` function uses brightness ($r \times 299 + g \times 587 + b \times 114 / 1000$, the luma formula) and saturation to determine neutrals, then hue angle for chromatic colors. This handles perceptual brightness correctly (human eyes are most sensitive to green, least to blue) but produces hard category boundaries — `#1a237e` (deep indigo) and `#283593` (slightly lighter indigo) both map to "navy" and score identically against a "navy" DNA entry, while `#1a237e` and `#1565c0` (cobalt) map to different buckets despite being perceptually similar.

Claude as a reasoning layer, not a scoring layer

Claude is used for tasks where the space of valid outputs is large and structured reasoning is required:

- Image attribute extraction (open-ended, image-grounded)
- Outfit assembly (combinatorial, aesthetically constrained)
- Agent intent parsing and conversation (ambiguous natural language)

Claude is explicitly NOT used for product scoring, confidence computation, or any loop that runs per-item — those are deterministic to keep costs predictable and outputs auditable.

Bottlenecks and Scaling Concerns

1. Image analysis throughput

`api/analyze.js` batches in groups of 3 via `Promise.all`. With 50 wardrobe photos, that's 17 sequential batches $\times$ ~1.5s each = ~25s of latency. Haiku is the right model choice here (fastest Claude tier), but the sequential batch structure limits parallelism. At higher image counts this becomes the primary user-facing latency.

2. Base64 image encoding memory

All images are fetched to the Vercel serverless function memory as base64 before being sent to Claude. A 1MB JPEG becomes ~1.37MB base64. With batches of 3 at ~1MB each: 4MB per batch, within Vercel's 1GB limit — but wardrobe photos from modern phones can be 5-10MB uncompressed. Client-side compression is the current mitigation, but it's not enforced server-side.

3. Vercel function cold starts on the agent

`api/agent.js` has `maxDuration: 60` but the agentic loop can take 4 full Claude round-trips plus SerpAPI. Cold starts on Vercel Edge can add 500ms–2s. The system compensates with a 110s race timeout.

4. SerpAPI latency and rate limits

Google Shopping searches average 2–4s per query. With 3 parallel category searches (e.g., tops + bottoms + shoes), the search step alone takes 3–4s. SerpAPI enforces per-plan request limits — at scale, this becomes a hard ceiling on concurrent users.

5. Style DNA is rebuilt entirely on each synthesis

`synthesize_style_dna` does a full recompute from all historical wardrobe/aspiration items on every call. With 100+ items this is fast (Supabase query + in-memory aggregation), but it's O(n) in item count and runs synchronously in the request path of `api/analyze.js`. At hundreds of items per user, this should become a background job.

Assumptions in the Pipeline

- 1. **Claude Vision hex codes are accurate.** The prompt asks Claude to "sample the actual image colors" but Claude is a language model, not a color sampler. Hex values are approximations. The pipeline treats them as ground truth.
- 2. **Product title contains style signals.** `inferFit` and `inferStyle` regex patterns assume that product titles are descriptive and English. Titles like "Item #38291 - W" produce null inference, which reduces scoring accuracy significantly.
- 3. **SerpAPI products are in-stock.** The search tool filters on price validity but not stock availability. Out-of-stock items reach the outfit builder.
- 4. **The aspiration gap is purely additive.** Gap is computed as styles in aspiration NOT in wardrobe. But a user with 50 coastal aspiration items and 1 coastal wardrobe item has no gap under this logic — the set membership check ignores relative magnitude.
- 5. **Feedback signals are correctly attributed.** `update_style_dna` receives item attributes as passed by the agent. If the agent misidentifies which item was rejected (e.g., user rejects an outfit but the agent logs the wrong item), the wrong attributes get penalized permanently.

Potential Failure Points

Failure	Impact	Current mitigation
Claude returns malformed JSON from image analysis	Item skipped, no attributes written	<code>skip_reason</code> field + try/catch in analyze loop
SerpAPI returns no results	Empty outfit result	Agent instructed to broaden query and retry once
Pinterest board is private	Analysis fails silently	Scraper returns explicit error state, agent surfaces it
<code>build_outfits</code> fuzzy match fails	Item dropped from outfit (product shows as null)	<code>.filter(i => i.product)</code> removes unresolved items
MAX_TURNS hit mid-pipeline	User gets a partial/fallback response	Recovery call: closes pending tool_use blocks, forces text reply
Supabase RLS blocks service key	All tool writes silently fail	Service key bypasses RLS — failure would surface as JS error
Hex-to-bucket collision	Wrong color score	No mitigation — by design limitation

Startup Framing

Core insight: Style preference is not something users can articulate — it's something you read. By analyzing what people already own and already save, the system extracts a preference model that is richer than anything a quiz or onboarding flow can capture. The Pinterest aspiration gap is particularly powerful: it's a pre-built, unprompted shopping list that the user created for themselves, passively, over months.

Why this is hard to replicate:

1. **The data moat compounds.** Every image analyzed, every swap made, every rejection logged refines the Style DNA. A competitor starting today with the same architecture has no profile history. The system gets meaningfully more accurate with 50 wardrobe items than with 5 — and that data is exclusive to each user's account.
2. **The WAT architecture keeps inference costs predictable.** Using Claude for reasoning and deterministic tools for execution means the marginal cost per recommendation is fixed and low — one agent loop, bounded at 4 turns, using Haiku. Competitors using frontier models for every step pay 10–20× more per request.
3. **The feedback loop closes inside the product.** Approval, rejection, swap, and delivery feedback all write back to the DNA. Most recommendation systems treat feedback as training data for a future model version. This system applies it on the next request.

Real-world value: For the user, recommendations feel increasingly inevitable rather than approximate. For the business, this is a personalization system that gets better with usage — classic retention flywheel mechanics applied to fashion.

Improve This System

Ordered by ROI (impact ÷ effort):

1. CIE76 Delta-E color distance (High impact / Low effort — 1 day)

Replace `hexToBucket()` comparison with perceptual color distance. Current bucket system collapses hundreds of distinct shades into ~20 names. Delta-E < 5 = visually indistinguishable; > 25 = clearly different. Implementation is ~20 lines of pure math, no dependencies, no API calls.

```
// Current: "#1a237e" and "#283593" both → "navy" → same score
// Delta-E: distance = 6.3 → partial match, not full match
```

Expected impact: Meaningfully improves color scoring precision for users with specific palette preferences (e.g., warm beige vs. cool grey — currently treated identically).

2. Fix the swap signal handler (High impact / Low effort — 4 hours)

The swap handler in `update_style_dna.js` is a documented no-op:

```
if (signalType === 'swap' && swapTarget) {
  // They moved toward swapTarget.fit_type — will be picked up next run
  // [nothing happens]
}
```

A swap is the strongest signal in the system — the user rejected one item and chose another simultaneously. The attribute delta is a directional preference vector. Writing `swapTarget` attributes to a

`swap_preferences` JSONB column and applying a `+12` bonus in the scorer would immediately improve recommendation quality for every user who has ever swapped.

3. CLIP embeddings + pgvector for visual similarity (Very high impact / High effort — 2 weeks)

The "similar to previously approved" and "similar to previously rejected" factors (+15/-30 in the scoring table) have **no implementation** — they're placeholder weights that never fire. Implementing them requires:

1. Enabling `pgvector` in Supabase
2. Generating CLIP embeddings for wardrobe items on analysis (`@xenova/transformers` runs in Vercel serverless, zero API cost)
3. Generating embeddings for product thumbnails at search time
4. Cosine similarity query: top-k wardrobe neighbors per product

This activates the most powerful potential signal in the system and captures visual gestalt that categorical labels miss entirely.

4. Async product image re-analysis (High impact / Medium effort — 1 week)

SerpAPI products arrive with `colors: null, fit_type: null, style_category: null`. All three are inferred from the product title via regex — the weakest link in the pipeline. A product titled "Women's Item - Sand/Multi" defeats all inference.

Build an async background job that for each new product seen:

1. Fetches the thumbnail
2. Runs `analyzeImageStyle()`
3. Caches the result in a `product_embeddings` table with a 7-day TTL

Subsequent scoring for the same product uses real attributes instead of title inference. Cache hit rate will be high — Google Shopping frequently returns the same product across different queries.

5. Temporal decay for negative signals (Medium impact / Medium effort — 1 day + schema migration)

Rejected colors and fits currently accumulate forever in `avoided_colors` and `explicit_dislikes.fits` with no expiry. A color rejected in winter may be acceptable in summer. Style evolves.

Add `signal_weight float` to `feedback_signals`. Recompute avoided attributes at write time using:
`weight = max(0.1, 1.0 - days_since_signal / 180)`

This converts the hard binary penalty system into a continuous decay, preventing stale signals from permanently suppressing items the user might now want.